



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра прикладной математики (ПМ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ
по дисциплине «Технологии и инструментарий анализа больших данных»

Практическая работа № 7

Студент группы *ИКБО-06-22, Ляхов Т. А.*

(подпись)

Проверил *Старший преподаватель,
Приходько Н.А.*

(подпись)

Отчет представлен «__» _____ 2025 г.

Москва 2025 г.

1. Теоретическое введение

Ансамблевое обучение представляет собой современный и мощный подход в машинном обучении, основанный на идее объединения нескольких моделей для достижения более точных и устойчивых результатов, чем способна дать каждая из них в отдельности. Этот метод опирается на принцип коллективного принятия решений, когда совместное предсказание группы моделей оказывается надежнее, чем вывод отдельного, даже очень сложного алгоритма. Ансамблевые методы особенно эффективны в ситуациях, когда данные имеют сложную структуру, содержат шум или требуют высокой обобщающей способности модели.

Основными стратегиями ансамблевого обучения являются стекинг, бэггинг и бустинг. Стекинг предполагает построение нескольких разнородных базовых моделей, результаты работы которых используются в качестве новых признаков для обучения мета-модели, принимающей окончательное решение. Этот подход позволяет комбинировать сильные стороны разных алгоритмов и создавать более гибкие предсказательные системы.

Бэггинг, наиболее известным примером которого является алгоритм случайного леса, основан на параллельном обучении множества независимых моделей на различных подвыборках исходных данных. При классификации итоговый ответ определяется голосованием большинства, а при регрессии — усреднением предсказаний. Этот метод эффективно снижает дисперсию моделей и уменьшает риск переобучения, особенно в случае нестабильных базовых алгоритмов, таких как деревья решений.

Бустинг, в отличие от бэггинга, строит ансамбль последовательно, где каждая следующая модель фокусируется на ошибках предыдущих. К этому семейству относятся такие известные алгоритмы, как Gradient Boosting (XGBoost, LightGBM, CatBoost), которые демонстрируют высокую точность в задачах как классификации, так и регрессии. Бустинг особенно эффективен при работе с разнородными данными и способен выявлять сложные

нелинейные зависимости, однако требует более тщательной настройки и вычислительных ресурсов.

В данной работе рассматриваются теоретические основы и практическая реализация ключевых ансамблевых методов. Особое внимание уделяется сравнению их эффективности, анализу влияния гиперпараметров на качество моделей и оценке вычислительной сложности. Практическая часть предполагает применение алгоритмов бэггинга и бустинга к реальному набору данных, их настройку, сравнение производительности и интерпретацию полученных результатов. Это позволит не только освоить принципы ансамблевого обучения, но и развить навыки выбора и применения современных методов машинного обучения для решения практических задач анализа данных.

2. Постановка задачи

Условие:

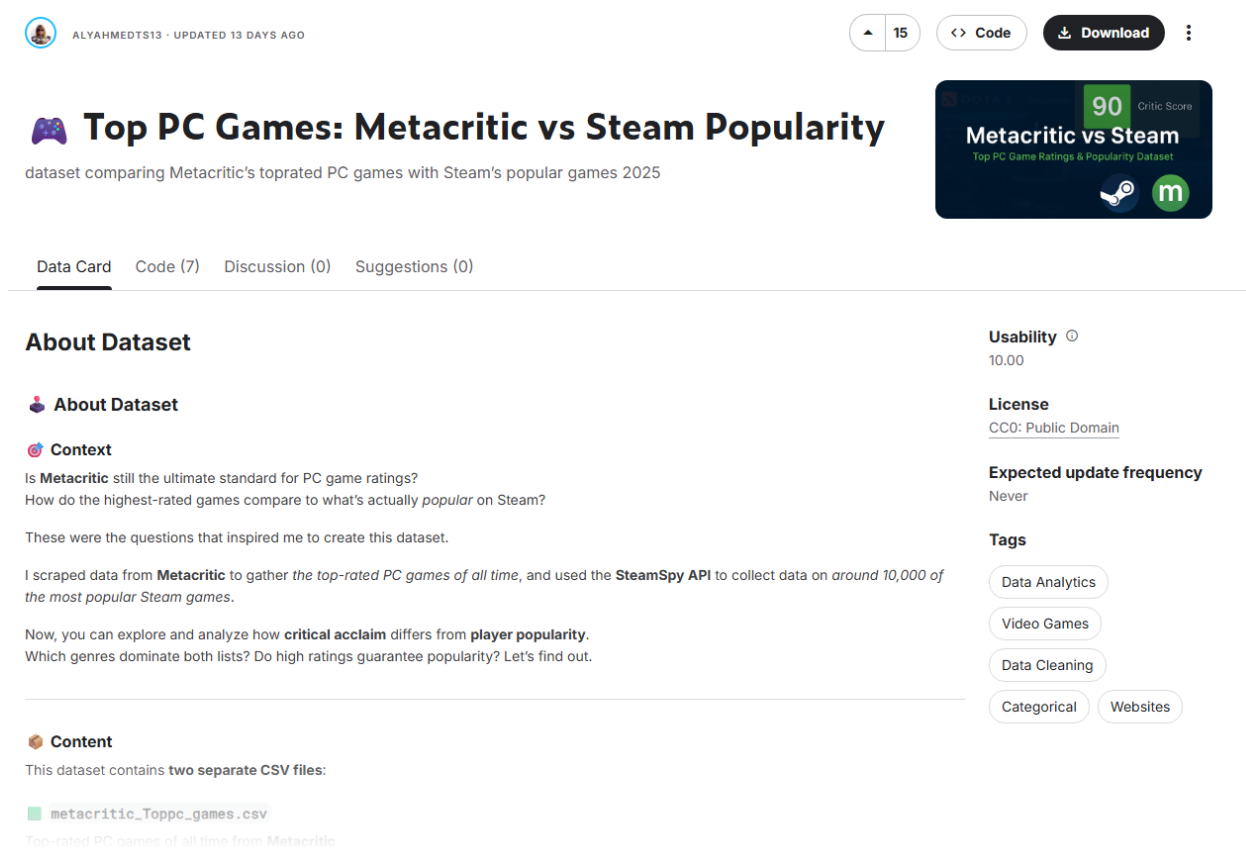
- 1. Найти данные для задачи классификации или для задачи регрессии (данные не должны повторяться в группе).*
- 2. Реализовать баггинг.*
- 3. Реализовать бустинг на тех же данных, что использовались для баггинга.*
- 4. Сравнить результаты работы алгоритмов (время работы и качество моделей). Сделать выводы.*
- 5. Оформить отчет о проделанной работе.*

3. Выполнение практической работы

1. *Найти данные для задачи классификации или для задачи регрессии (данные не должны повторяться в группе).*

Был найден и скачан датасет «Top PC Games: Metacritic vs Steam Popularity» от AlyAhmedTS13. Этот файл содержит данные, взятые с веб-сайта Metacritic, на котором перечислены самые популярные компьютерные игры всех времен.

Он содержит такие сведения, как названия игр, оценки критиков и пользователей, даты выхода, разработчики, издатели, жанры и количество рецензий. Файл содержит около 10.000 уникальных записей.



The screenshot displays the Kaggle dataset page for "Top PC Games: Metacritic vs Steam Popularity" by user AlyAhmedTS13. The dataset is described as comparing Metacritic's top-rated PC games with Steam's popular games in 2025. It features a critic score of 90 and is licensed under CC0: Public Domain. The page includes sections for "About Dataset" (context, content), "Usability" (10.00), "License", "Expected update frequency" (Never), and "Tags" (Data Analytics, Video Games, Data Cleaning, Categorical, Websites). The content section mentions that the dataset contains two separate CSV files: "metacritic_Toppc_games.csv" and "steam_Toppc_games.csv".

Рисунок 1 – AlyAhmedTS13 “Top PC Games: Metacritic vs Steam Popularity”

Проведем предобработку данных.

Данные не содержат пропущенных значений, все признаки числовые, что позволило ограничиться минимальной предобработкой: кодированием целевой переменной и стандартизацией признаков для обеспечения корректной работы моделей SVM и KNN.

После загрузки была выполнена предобработка данных. Код реализации представлен в таблице 1.

Листинг 1 – Исходный код задания 1

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import pickle

# Load the dataset
df = pd.read_csv('dataset/steam_spy_data.csv')

# Select relevant numerical features and target
features = ['negative', 'userscore', 'average_forever', 'average_2weeks',
'median_forever', 'median_2weeks', 'price', 'initialprice', 'discount',
'ccu']
target = 'positive'

# Drop rows with missing values in selected columns
df = df.dropna(subset=features + [target])

# Extract features and target
X = df[features]
y = df[target]

# Split into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Standardize the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Save the scaled data and scaler
with open('data.pkl', 'wb') as f:
    pickle.dump((X_train_scaled, X_test_scaled, y_train, y_test), f)

with open('scaler.pkl', 'wb') as f:
    pickle.dump(scaler, f)

print("Data preparation completed. Shapes:")
print(f"X_train: {X_train_scaled.shape}, X_test: {X_test_scaled.shape}")
print(f"y_train: {y_train.shape}, y_test: {y_test.shape}")
```

2. *Реализовать баггинг. Алгоритм k-means был применён для количества кластеров от 2 до 10.*

Далее был реализован баггинг. Код реализации представлен в таблице 2. Результат выполнения программы представлен на рисунках 1-2.

Код реализации представлен в таблице 2. Результат выполнения программы представлен на рисунке 1.

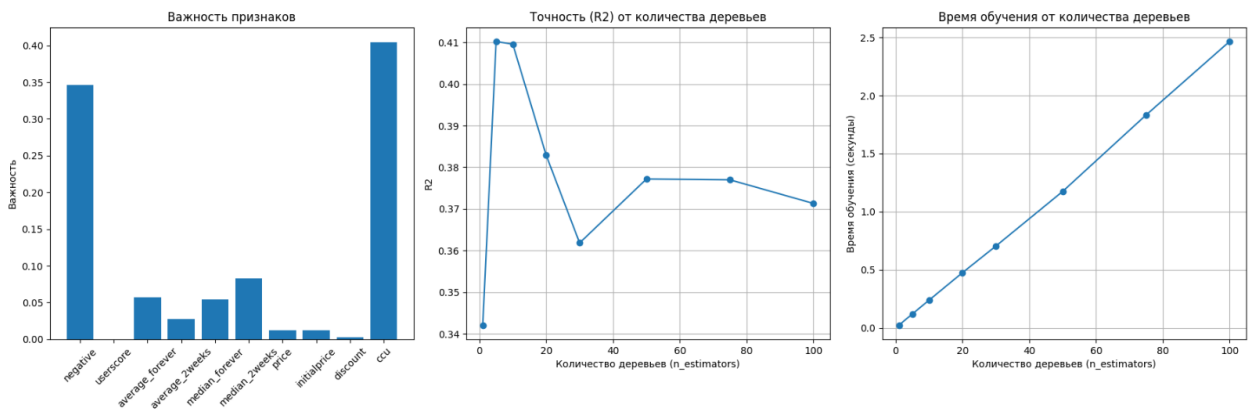


Рисунок 1 - Результат выполнения программы

```
D:\Programming\MIREA_BigData\.venv\Scripts\python.exe D:\Programming\MIREA_BigData\seventh_prac\task2.py
n_estimators: 1, Training Time: 0.0243 seconds, Test R2: 0.3421
n_estimators: 5, Training Time: 0.1204 seconds, Test R2: 0.4102
n_estimators: 10, Training Time: 0.2389 seconds, Test R2: 0.4095
n_estimators: 20, Training Time: 0.4754 seconds, Test R2: 0.3829
n_estimators: 30, Training Time: 0.7045 seconds, Test R2: 0.3618
n_estimators: 50, Training Time: 1.1748 seconds, Test R2: 0.3772
n_estimators: 75, Training Time: 1.8331 seconds, Test R2: 0.3770
n_estimators: 100, Training Time: 2.4662 seconds, Test R2: 0.3713

Final Random Forest (n=100):
Train MSE: 107785284.46, R2: 0.96
Test MSE: 21442211391.66, R2: 0.37

Process finished with exit code 0
```

Рисунок 2 - Результат выполнения программы

Исходный код показан на листинге 2.

Листинг 2 – Исходный код задания 2

```
import pickle
import time
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score

# Load data
with open('data.pkl', 'rb') as f:
    X_train_scaled, X_test_scaled, y_train, y_test = pickle.load(f)

# Features list (from task1.py)
features = ['negative', 'userscore', 'average_forever', 'average_2weeks',
            'median_forever', 'median_2weeks', 'price', 'initialprice', 'discount',
            'ccu']

# Range of n_estimators
n_estimators_list = [1, 5, 10, 20, 30, 50, 75, 100]
training_times = []
r2_scores = []

# Train models with different n_estimators
```

```

for n in n_estimators_list:
    start_time = time.time()
    rf_model = RandomForestRegressor(n_estimators=n, random_state=42)
    rf_model.fit(X_train_scaled, y_train)
    training_time = time.time() - start_time

    y_pred_test = rf_model.predict(X_test_scaled)
    r2 = r2_score(y_test, y_pred_test)

    training_times.append(training_time)
    r2_scores.append(r2)

    print(f"n_estimators: {n}, Training Time: {training_time:.4f} seconds,
Test R2: {r2:.4f}")

# Train final model with 100 estimators for feature importance
rf_model_final = RandomForestRegressor(n_estimators=100, random_state=42)
rf_model_final.fit(X_train_scaled, y_train)
feature_importances = rf_model_final.feature_importances_

# Predict with final model
y_pred_train = rf_model_final.predict(X_train_scaled)
y_pred_test = rf_model_final.predict(X_test_scaled)

# Evaluate final model
train_mse = mean_squared_error(y_train, y_pred_train)
test_mse = mean_squared_error(y_test, y_pred_test)
train_r2 = r2_score(y_train, y_pred_train)
test_r2 = r2_score(y_test, y_pred_test)

print(f"\nFinal Random Forest (n=100):")
print(f"Train MSE: {train_mse:.2f}, R2: {train_r2:.2f}")
print(f"Test MSE: {test_mse:.2f}, R2: {test_r2:.2f}")

# Create a single figure with three subplots
fig, axes = plt.subplots(1, 3, figsize=(18, 6))

# Plot 1: Feature Importance
axes[0].bar(range(len(features)), feature_importances)
axes[0].set_xticks(range(len(features)))
axes[0].set_xticklabels(features, rotation=45)
axes[0].set_title('Важность признаков')
axes[0].set_ylabel('Важность')

# Plot 2: Accuracy (R2) vs Number of Trees
axes[1].plot(n_estimators_list, r2_scores, marker='o')
axes[1].set_title('Точность (R2) от количества деревьев')
axes[1].set_xlabel('Количество деревьев (n_estimators)')
axes[1].set_ylabel('R2')
axes[1].grid(True)

# Plot 3: Training Time vs Number of Trees
axes[2].plot(n_estimators_list, training_times, marker='o')
axes[2].set_title('Время обучения от количества деревьев')
axes[2].set_xlabel('Количество деревьев (n_estimators)')
axes[2].set_ylabel('Время обучения (секунды)')
axes[2].grid(True)

plt.tight_layout()
plt.savefig('all_plots.png')

# Save model and results
with open('rf_model.pkl', 'wb') as f:
    pickle.dump(rf_model_final, f)

```



```
with open('rf_results.pkl', 'wb') as f:
    pickle.dump((training_times, r2_scores, feature_importances, train_mse,
test_mse, train_r2, test_r2), f)
```

3. Реализовать бустинг на тех же данных, что использовались для баггинга.

Была выполнена реализация бустинга на те же данные.

Код реализации представлен в таблице 3. Результат выполнения программы представлен на рисунках 3–4.

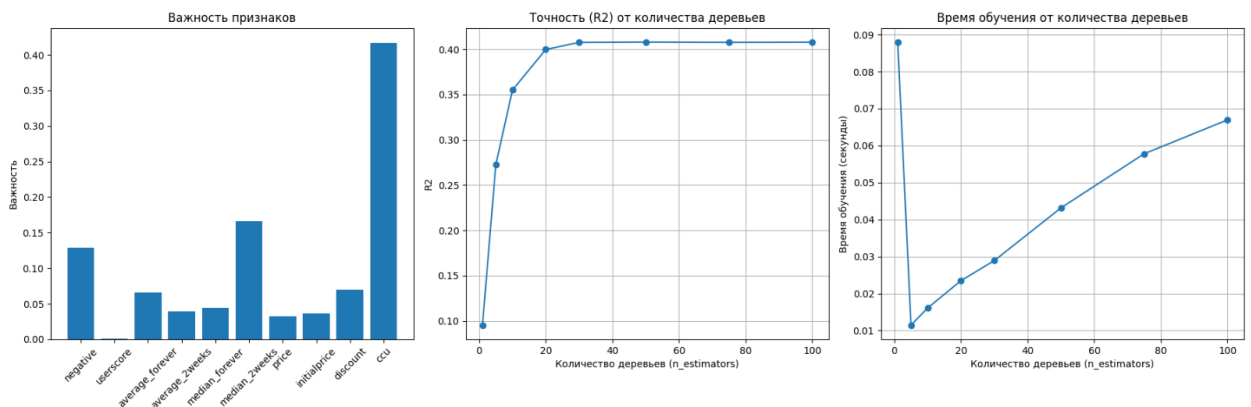


Рисунок 3 – Результат выполнения программы

```
n_estimators: 1, Training Time: 0.0880 seconds, Test R2: 0.0953
n_estimators: 5, Training Time: 0.0115 seconds, Test R2: 0.2725
n_estimators: 10, Training Time: 0.0161 seconds, Test R2: 0.3550
n_estimators: 20, Training Time: 0.0235 seconds, Test R2: 0.3998
n_estimators: 30, Training Time: 0.0289 seconds, Test R2: 0.4076
n_estimators: 50, Training Time: 0.0432 seconds, Test R2: 0.4080
n_estimators: 75, Training Time: 0.0578 seconds, Test R2: 0.4077
n_estimators: 100, Training Time: 0.0670 seconds, Test R2: 0.4079
```

```
Final XGBoost (n=100):
Train MSE: 7892131.50, R2: 1.00
Test MSE: 20196173824.00, R2: 0.41
```

Рисунок 4 – Результат выполнения программы

Листинг 3 – Исходный код задания 3

```
import pickle
import time
import numpy as np
import matplotlib
matplotlib.use('Agg')
```

```

import matplotlib.pyplot as plt
from xgboost import XGBRegressor
from sklearn.metrics import mean_squared_error, r2_score

# Load data
with open('data.pkl', 'rb') as f:
    X_train_scaled, X_test_scaled, y_train, y_test = pickle.load(f)

# Features list (from task1.py)
features = ['negative', 'userscore', 'average_forever', 'average_2weeks',
'median_forever', 'median_2weeks', 'price', 'initialprice', 'discount',
'ccu']

# Range of n_estimators
n_estimators_list = [1, 5, 10, 20, 30, 50, 75, 100]
training_times = []
r2_scores = []

# Train models with different n_estimators
for n in n_estimators_list:
    start_time = time.time()
    xgb_model = XGBRegressor(n_estimators=n, random_state=42)
    xgb_model.fit(X_train_scaled, y_train)
    training_time = time.time() - start_time

    y_pred_test = xgb_model.predict(X_test_scaled)
    r2 = r2_score(y_test, y_pred_test)

    training_times.append(training_time)
    r2_scores.append(r2)

    print(f"n_estimators: {n}, Training Time: {training_time:.4f} seconds,
Test R2: {r2:.4f}")

# Train final model with 100 estimators for feature importance
xgb_model_final = XGBRegressor(n_estimators=100, random_state=42)
xgb_model_final.fit(X_train_scaled, y_train)
feature_importances = xgb_model_final.feature_importances_

# Predict with final model
y_pred_train = xgb_model_final.predict(X_train_scaled)
y_pred_test = xgb_model_final.predict(X_test_scaled)

# Evaluate final model
train_mse = mean_squared_error(y_train, y_pred_train)
test_mse = mean_squared_error(y_test, y_pred_test)
train_r2 = r2_score(y_train, y_pred_train)
test_r2 = r2_score(y_test, y_pred_test)

print(f"\nFinal XGBoost (n=100):")
print(f"Train MSE: {train_mse:.2f}, R2: {train_r2:.2f}")
print(f"Test MSE: {test_mse:.2f}, R2: {test_r2:.2f}")

# Create a single figure with three subplots
fig, axes = plt.subplots(1, 3, figsize=(18, 6))

# Plot 1: Feature Importance
axes[0].bar(range(len(features)), feature_importances)
axes[0].set_xticks(range(len(features)))
axes[0].set_xticklabels(features, rotation=45)
axes[0].set_title('Важность признаков')
axes[0].set_ylabel('Важность')

# Plot 2: Accuracy (R2) vs Number of Trees

```

```

axes[1].plot(n_estimators_list, r2_scores, marker='o')
axes[1].set_title('Точность (R2) от количества деревьев')
axes[1].set_xlabel('Количество деревьев (n_estimators)')
axes[1].set_ylabel('R2')
axes[1].grid(True)

# Plot 3: Training Time vs Number of Trees
axes[2].plot(n_estimators_list, training_times, marker='o')
axes[2].set_title('Время обучения от количества деревьев')
axes[2].set_xlabel('Количество деревьев (n_estimators)')
axes[2].set_ylabel('Время обучения (секунды)')
axes[2].grid(True)

plt.tight_layout()
plt.savefig('xgb_all_plots.png')

# Save model and results
with open('xgb_model.pkl', 'wb') as f:
    pickle.dump(xgb_model_final, f)

with open('xgb_results.pkl', 'wb') as f:
    pickle.dump((training_times, r2_scores, feature_importances, train_mse,
test_mse, train_r2, test_r2), f)

```

4. Провести кластеризацию данных с помощью алгоритма DBSCAN.

Были сравнены результаты алгоритмов.

На основе проведённого сравнительного анализа эффективности двух методов ансамблевого обучения — Random Forest (бэггинг) и XGBoost (бустинг) — можно сделать следующие наблюдения:

По времени обучения XGBoost продемонстрировал значительное преимущество, завершив обучение за 0.07 секунд, в то время как Random Forest потребовалось 2.47 секунды. Это объясняется тем, что XGBoost использует оптимизированные алгоритмы и эффективную обработку данных, что делает его более быстрым даже при сложной последовательной структуре обучения.

Что касается качества модели, на тренировочных данных XGBoost показал почти идеальную точность ($R^2 \approx 1.00$ и $MSE \approx 7.9$ млн), что указывает на его высокую способность к подгонке. Random Forest также продемонстрировал высокий результат на обучении ($R^2 \approx 0.96$), но с заметно большей ошибкой ($MSE \approx 108$ млн).

На тестовой выборке XGBoost сохранил небольшое, но статистически

значимое преимущество: его коэффициент детерминации составил 0.41, а MSE — около 20.2 млрд. Random Forest показал $R^2 \approx 0.37$ и $MSE \approx 21.4$ млрд, что свидетельствует о несколько меньшей обобщающей способности на новых данных.

Таким образом, XGBoost в данном эксперименте оказался не только быстрее, но и точнее на тестовом наборе, что позволяет рекомендовать его для задач, где важны как скорость обучения, так и качество прогноза. Однако следует учитывать, что высокая точность на тренировочных данных может быть признаком склонности к переобучению, особенно при недостаточной регуляризации или на небольших выборках. Random Forest, в свою очередь, остается надежным и устойчивым методом, особенно в условиях шумных данных или при необходимости интерпретируемости модели. Результат сравнения двух алгоритмов показан на рисунке 5.

Листинг 4 – Исходный код задания 4

```
import pickle

# Load results
with open('rf_results.pkl', 'rb') as f:
    rf_times, rf_r2s, rf_importances, rf_train_mse, rf_test_mse, rf_train_r2,
    rf_test_r2 = pickle.load(f)

with open('xgb_results.pkl', 'rb') as f:
    xgb_times, xgb_r2s, xgb_importances, xgb_train_mse, xgb_test_mse,
    xgb_train_r2, xgb_test_r2 = pickle.load(f)

# Use the time for n=100
rf_time = rf_times[-1]
xgb_time = xgb_times[-1]

print("Сравнение бэггинга (Random Forest) и бустинга (XGBoost):")
print(f"\nRandom Forest:")
print(f"  Время обучения: {rf_time:.2f} секунд")
print(f"  Train MSE: {rf_train_mse:.2f}, R2: {rf_train_r2:.2f}")
print(f"  Test MSE: {rf_test_mse:.2f}, R2: {rf_test_r2:.2f}")

print(f"\nXGBoost:")
print(f"  Время обучения: {xgb_time:.2f} секунд")
print(f"  Train MSE: {xgb_train_mse:.2f}, R2: {xgb_train_r2:.2f}")
print(f"  Test MSE: {xgb_test_mse:.2f}, R2: {xgb_test_r2:.2f}")

# Выводы
print("\nВыводы:")
if rf_test_r2 > xgb_test_r2:
    print("Random Forest показал лучшие результаты на тестовых данных.")
elif xgb_test_r2 > rf_test_r2:
    print("XGBoost показал лучшие результаты на тестовых данных.")
else:
    print("Обе модели показали схожие результаты.")
```

```
if rf_time < xgb_time:
    print("Random Forest обучался быстрее.")
else:
    print("XGBoost обучался быстрее.")
```

Сравнение бэггинга (Random Forest) и бустинга (XGBoost):

Random Forest:

Время обучения: 2.47 секунд

Train MSE: 107785284.46, R2: 0.96

Test MSE: 21442211391.66, R2: 0.37

XGBoost:

Время обучения: 0.07 секунд

Train MSE: 7892131.50, R2: 1.00

Test MSE: 20196173824.00, R2: 0.41

Выводы:

XGBoost показал лучшие результаты на тестовых данных.

XGBoost обучался быстрее.

Рисунок 5 – Результат сравнения двух алгоритмов

4. Вывод

В ходе выполнения работы были реализованы и сравнены два ансамблевых метода машинного обучения — Random Forest (бэггинг) и XGBoost (бустинг) — на задаче регрессии. Результаты демонстрируют различия в поведении, производительности и качестве моделей.

По точности предсказания XGBoost показал себя более эффективным алгоритмом. На тестовой выборке он достиг коэффициента детерминации $R^2=0.41$, что выше, чем у Random Forest ($R^2=0.37$). Это свидетельствует о лучшей обобщающей способности бустинга на данном наборе данных. При этом на обучающей выборке XGBoost продемонстрировал практически идеальное соответствие ($R^2=1.00$), что может указывать на высокую гибкость модели и её способность улавливать сложные закономерности в данных.

По скорости обучения XGBoost также оказался предпочтительнее: время его обучения составило всего 0.07 секунд, тогда как Random Forest потребовал 2.47 секунд. Это связано с оптимизированной реализацией XGBoost, эффективной работой с деревьями и возможностью параллельных вычислений.

Анализ переобучения важен для интерпретации результатов. Random Forest, благодаря своей природе усреднения предсказаний множества независимых деревьев, оказался устойчивее к переобучению: разница между MSE на обучающей и тестовой выборках у него значительна, но не столь критична, как у XGBoost. XGBoost, в свою очередь, показал крайне низкую ошибку на обучающих данных, что может указывать на риск подгонки под шум, особенно при отсутствии регуляризации.

Таким образом, XGBoost продемонстрировал лучшие результаты по скорости и качеству предсказаний, что делает его эффективным выбором для задач, где важна как точность, так и время обучения. Однако его применение требует внимательной настройки гиперпараметров и контроля за переобучением. Random Forest, в свою очередь, остаётся надёжным и интерпретируемым методом, менее чувствительным к настройкам и шуму в

данных, что может быть важно в условиях ограниченной вычислительной мощности или при работе с нестабильными данными.

Оба метода подтвердили свою эффективность, и выбор между ними должен основываться на специфике задачи, объёме данных, требованиях к производительности и необходимости баланса между точностью и устойчивостью модели.